A5 学生版讲义 · 打包双 jar + 局域网上线 + 全流程复盘

> **课程**:CS 实训(C/S 模式 Java 图形界面开发)· Vibe Coding 单元⑤(Part A 收官) **用法**:课前预习、课后复习都能看。内容精炼,配合课堂消化。 **今天做三件收官的事**:打包双 jar、局域网多机上线、复盘 Vibe Coding 全流程。(AI 图书助手是可选 / 挑战。) **今天先补两块基础**:前四单元你都在 IDE 里点"运行",从没自己打过包、没让别的电脑连进来。所以下面先有 **jar 零基础** 和 **局域网零基础** 两块速记,读懂它们再看操作就不虚。

一、C/S 的"上线"是什么:和网页不是一回事

网页(B/S)上线,是部署到服务器让**浏览器**访问。我们这门课是 **C/S 桌面程序**,"上线"是另一回事:

- **打包**:把 Java 代码打成**可执行 jar**,别人不装 IDE、有 Java 运行时就能双击 / `java -jar` 跑。
- **局域网多机**:**一台电脑当服务器**(跑 MySQL + 服务端 jar),**多台电脑当客户端**(跑客户端 jar),客户端登录窗填服务器电脑的 **IP**,连同一个服务端。

> 这正是我们档 B 三段式的价值:数据库和业务只在服务端一台,客户端只管界面、不碰库。写得再好,跑不起来给人用等于零——这就是软件的"最后一公里"。

二、⭐ jar 零基础速记(打包前先认全这几个词)

词	一句话	生活类比
jar	把一堆编译出来的 `.class` 打成的**一个文件**,谁拿到都能跑	zip 压缩包 / 安装包(jar 本质就是 zip)
`javac`	把你写的 `.java`(源码)编译成 `.class`(能跑的字节码),jar 里装的就是 `.class`	图纸 → 零件 → 装箱
清单 Main-Class	jar 里一行配置,写明 `java -jar` 时**从哪个类的 `main` 启动**	一栋楼的**大门**,告诉别人从哪进
classpath	告诉 Java"要用的类 / jar 在哪几个地方找",找不到报 `ClassNotFoundException`	"去哪几个书架找书"的清单
fat jar	把依赖(MySQL 驱动)**一起打进 jar**,一个包就能连库	把电池也装进玩具盒,拆开就能玩

从 `.java` 到能发的一条链:`.java`(你写的)——`javac` 编译——▶ `.class`(能跑的)——打包——▶ `.jar`(能发的)。

最小命令示例(认全三条命令):

```
javac -encoding UTF-8 -d out Hello.java      # ① 编译:.java → .class
jar cfe hello.jar Hello -C out .             # ② 打包:c 创建 / f 文件名 / e 入口类
java -jar hello.jar                          # ③ 运行:按清单 Main-Class 启动
```

> 记牢 `jar cfe` = **c**reate 创建 / **f**ile 文件名 / **e**ntry 入口类。图书馆的打包脚本,就是在这三条上多做两件事:编译时挂驱动、打服务端包时合并驱动。

三、为什么打两个 jar

系统本来就是两个独立运行的程序,所以打两个包,各跑各的:

jar	是什么	谁运行	含 MySQL 驱动吗
`library-server.jar`	服务端(监听 9000 + 多线程 + 连库)	服务器电脑(一台)	**要** → 打成 **fat jar**
`library-client.jar`	客户端(纯 Swing 界面)	客户端电脑(多台)	**不要**(客户端不直连库)

答案版打出来:`library-server.jar` 约 **2.5MB**(含驱动,胖包),`library-client.jar` 约 **77KB**(纯 Swing,瘦包)。一大一小,一眼看出谁带了驱动。

四、⭐ fat jar 内置驱动 + 去签名(打包最经典的坑)

- **fat jar** = 把依赖(MySQL 驱动)一起打进 jar。**服务端要连库,所以要 fat jar**;客户端不连库,普通 jar 就行。
- ⚠️ **去签名**:MySQL 官方驱动自带数字签名文件(`META-INF/` 下的 `*.SF` / `*.RSA` / `*.DSA`)和自己的 `MANIFEST.MF`。你把驱动内容合并进自己的 jar 后,签名和内容对不上,一跑就报**签名校验错**(`SecurityException` / `digest error`)。**所以合并时必须删掉这些签名文件和 `META-INF/versions`**,只保留 `META-INF/services/java.sql.Driver`(它让 `DriverManager` 自动找到驱动)。
  - 生活类比:签名文件像原厂**防伪封条**。你拆开驱动、重新装成新包,旧封条对不上新包,安检(JVM)就拦。解法是**撕掉旧封条**(删签名)。

`build-jars.sh` 的三步(看懂即可,课上会跑):

```
# ① 编译全部源码到 out/(编译时把驱动放进 classpath)
javac -encoding UTF-8 -cp "lib/mysql-connector-j-8.0.33.jar" -d out
@/tmp/_srcs.txt
# ② 客户端 jar(纯 Swing,入口登录窗)
jar cfe dist/library-client.jar client.ui.LoginFrame -C out .
# ③ 服务端 fat jar(合并驱动 → 删签名 → 打包,入口监控窗口)
cp -r out/* stage/ && ( cd stage && unzip -oq "../lib/mysql-connector-j-
8.0.33.jar" )
rm -f stage/META-INF/*.SF stage/META-INF/*.RSA stage/META-INF/*.DSA stage/META-
INF/MANIFEST.MF
rm -rf stage/META-INF/versions
jar cfe dist/library-server.jar server.ui.ServerMonitorFrame -C stage .
```

> `jar cfe`:`c`=创建、`f`=文件名、`e`=入口类。所以服务端 jar 双击进监控台、客户端 jar 双击进登录窗。第 ③ 步那两行 `rm` 就是"去签名";`unzip 驱动.jar` 就是把驱动当 zip 解开。

启动(命令与答案版一致):

```
java -jar dist/library-server.jar      # 先起服务端,弹出监控台窗口
java -jar dist/library-client.jar      # 再起客户端,可开多个
```

五、⭐ 局域网零基础速记(让别的电脑连进来前先认全)

词	一句话	生活类比(一栋楼)
局域网	连在**同一路由器 / WiFi 热点**下的一群机器,能互相直接找到、说话	同一**栋楼**,房间能串门、打内线
IP 地址	每台机器的门牌号(如 `192.168.1.10`);`127.0.0.1`(localhost)= **我自己这台**	房间**房间号**
端口	找到机器后再找**具体哪个程序**;服务端用 **9000**,MySQL 用 3306	房间里的**电话分机号**
防火墙	门口"门卫",默认拦外面打进来的连接;要放行入站 9000 别人才连得进	楼下**门卫**,不放行进不来

查一台电脑的局域网 IP:Windows `ipconfig` 找 IPv4(如 `192.168.1.10`);Mac `ifconfig | grep 192`。

为什么不用改代码就支持局域网:服务端 `new ServerSocket(9000)` 默认绑 `0.0.0.0`(本机所有网卡都听),本机连、局域网连它都接。从单机切局域网,**代码一行不改**,只需"客户端填对 IP + 服务器放行防火墙"。

六、局域网怎么跑通:四件事

```
[客户端 A] ─┐
[客户端 B] ─┼─ 同一 WiFi ──→ [服务器电脑:MySQL + library-server.jar]
[客户端 C] ─┘    登录窗填服务器 IP + 端口 9000    防火墙放行入站 TCP 9000
```

#	要做的事	怎么做
①	一台服务器电脑	只在这台跑 MySQL + 服务端 jar,`init.sql` 也只在这台建
②	查服务器 IP	Windows `ipconfig` 找 IPv4(如 `192.168.1.10`);Mac `ifconfig | grep 192`
③	客户端填 IP	客户端登录窗**服务器地址填服务器电脑的 IP**(不是 `127.0.0.1`)、端口 `9000`
④	防火墙 + 同网	服务器电脑**防火墙放行入站 TCP 9000**;所有电脑连**同一 WiFi**

> 🔑 最容易踩的两个坑:① 客户端登录窗还填 `127.0.0.1`(那是它自己,连不到别人);② 服务器电脑防火墙把 9000 拦了(Windows Defender 常默认拦)。IP 填对 + 防火墙放行,九成问题就没了。

七、⭐ 并发防超借:上线最值得演示的一幕

开 2~3 个客户端,**同时借同一本只剩最后一本的书**(答案版《万历十五年》初始 `stock=1`):

- **预期**:只有一个提示"借阅成功",其余"库存不足,借阅失败"——**不会超借**。
- **为什么**:服务端 `BorrowService.borrow()` 在**一个事务**里 `UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0`,按**影响行数**判成败(为 0 说明没库存);扣库存 + 校验上限 + 写记录三步全成才 `commit()`,任一失败 `rollback()`。
- **答辩会问**:多台客户端同时借为什么不出错?**因为业务和事务集中在服务端一处把关**——这就是档 B 三段式的价值。这个演示要多台客户端,单机凑不齐,局域网上线才做得成。

★ 收官验收:端到端回归十站清单 + PRD 覆盖率自查(上线前最后一道闸)

> 打包了、局域网连通了、并发也不超借——但这只是"单点都对"。上线前再走两步:① 一条真实业务链路**从头拉通**(端到端回归);② 对照 PRD **打钩盘点**做出来几成(覆盖率自查)。练的都是 **[信任但验证]**。

端到端回归 · 十站清单(投屏一条条勾)

一条链子:**注册 → 审核通过 → 登录 → 查书 → 借书 → 续借 → 还书(制造逾期→罚款)→ 管理端统计报表看到 → 审计日志查到 →(并发借最后一本只成一个)**。十站,一站都不能断。

站	验收点
1	注册 test01 后,库里 `status=pending`
2	管理端审核通过后 `status` 变 `active`
3	审核前登录被拒 / 审核后登录成功(负向 + 正向都验)
4	查书能查到、可借数正确
5	借书成功且库存 -1、`t_borrow` 多一条
6	续借成功、应还日期后延;再续一次超上限被拒
7	还书后 `t_fine` 有一条,金额 = 逾期天数×0.5(手算核对)
8	管理端统计报表这笔进了聚合(借阅数 / 罚款额都对)
9	审计日志 `LOG_LIST` 查到 test01 这一串 + 管理员那次审核
并发	两客户端同借最后一本,只成一个

- **制造逾期不用等真到期**:借完后在服务器上手动把应还日期改早,再去还书就算出罚款:

```
UPDATE t_borrow SET due_time = DATE_SUB(CURDATE(), INTERVAL 1 DAY)
WHERE user_id = (SELECT id FROM t_user WHERE username='test01') AND
status='借出';
```

- **某站断了 → 定位是哪一层**(回归最练人的地方):连不上 = **网络层**(IP / 防火墙 / 服务端没起)、审核后登录不了 = **认证 / 数据层**、查书为空 = **数据层**、借书失败但库存有 = **业务 / 事务层**(`WHERE stock>0` 影响行数)、没罚款 / 金额不对 = **业务规则层**、统计看不到 = **聚合 SQL 层**、审计查不到 = **日志埋点层**。
- **进阶练法**:让 AI 教"主动制造故障再定位"——故意 IP 填错 / 关掉服务器 MySQL / 不审核就登录 / 借库存为 0 的书,各看什么现象、去哪层查;再加一条"回滚清理"把 test01 的借阅 / 罚款 / 日志删掉,别污染演示数据。

PRD 覆盖率自查(对照图纸打钩)

回归验"跑通的链子对不对";覆盖率自查换个角度:**当初 PRD(`.trae/rules`)写要做的功能,做出来几成?** 让 AI 生成三大类(① 协议命令 ② 窗体界面 ③ 业务规则)"PRD 功能 → 我做了吗"打钩表,每行标 **必做 / 加分 + 完成度(□未做 □半成 □完成)+ 一句验收标准**,逐条真实自评。再让 AI **当审查员反审**:挑必做缺口排序、每个缺口给一句"补它的最小 prompt"、给必做项完成度百分比(别报喜不报忧)。到 80% 以上基本立起来了;不到也拿到了"缺口 + 补丁 prompt",课后照补。

prompt 速查(端到端回归 + 覆盖率自查)

⑤ 生成端到端回归测试用例表

```
我要给这个 C/S 图书馆做一次端到端回归测试(局域网上线后验收)。请帮我生成一份"端到端测试用例表",
覆盖这条完整业务链路(一站都不要漏):新读者注册 → 管理员审核通过 → 该读者登录 → 查书 → 借一本书
→ 续借 → 还书(制造逾期,产生罚款)→ 切管理端看借阅统计报表里有这笔 → 操作审计日志里查得到。
每条用例给:序号、操作步骤、在哪一端做、协议命令
(REGISTER|../USER_APPROVE|../LOGIN|../BOOK_QUERY|../
BORROW|../RENEW|../RETURN|..)、预期结果、一个"是否通过"勾选框 □。再单列负向用例:① 审核前
就登录→应被拒;
② 两客户端并发借最后一本→只成一个;③ 续借超上限→应被拒。用中文表格输出,我要投屏照着勾。
```

⑥ PRD 覆盖率自查打钩表 + 让 AI 反审缺口

```
这是我这个 C/S 图书馆项目当初的 PRD(见 .trae/rules)。请帮我生成一张"PRD 功能→我做出来了
吗"自查打钩表,
分三大类列全:① 协议命令
(LOGIN/REGISTER/BOOK_QUERY/BORROW/RETURN/RENEW/USER_APPROVE/STATS_*/LOG_LIST/分类
CRUD…);
② 窗体界面(登录/注册/读者端查借还/管理端图书CRUD/用户审核/统计面板/服务端监控窗…);
③ 业务规则(借书事务防超借/续借上限/逾期罚款/用户审核三态/权限区分…)。每行给:功能项、必做/加分
项、
完成度勾选框(□未做 □半成 □完成)、一句验收标准。用中文表格,我要照着自评。
填好后请你当审查员:① 从我标"未做/半成"里挑必做 vs 加分排序;② 每个缺口给一句"补它的最小
prompt";
③ 给必做项完成度百分比,别报喜不报忧。
```

八、(可选 / 挑战)AI 图书助手 + Function Calling

> ⚠️ 可选,需要**你自己的大模型 key**(platform.deepseek.com 申请)。没 key 跳过,不影响本单元验收。

大模型很聪明,但**不知道你图书馆有哪些书**(书在你数据库里)。解决办法叫 **Function Calling**:你给大模型一个工具 `searchBooks`,它遇到查书问题时**决定调用它**,你的程序查库、把真实结果给它,它再组织成人话。

```
① 读者问 → 客户端把问题 + 工具说明发给大模型
② 大模型:要查库 → "请调用 searchBooks(keyword='Java')"
③ 客户端:发 BOOK_QUERY|Java|0 给服务端 → 服务端查 t_book → 真实藏书发回大模型
④ 大模型组织成人话回答 → 显示在聊天面板
```

- 大模型负责"理解 + 说话",你的系统负责"查真实数据"——配合起来既聪明又不骗人。
- AI 助手加在**客户端**,但查库仍走 Socket 发 `BOOK_QUERY` 给服务端。**客户端不碰库,档 B 没被破坏。**
- system 提示词要强调"只能根据查到的真实藏书回答,不许编造"——这是给 AI 立规矩(信任但验证)。

九、prompt 速查(可复制)

① 打包双 jar

```
教我把这个 C/S 项目打成两个可执行 jar:
- 客户端 jar(纯 Swing,Main-Class=client.ui.LoginFrame);
- 服务端 fat jar(把 MySQL 驱动合并进去、去掉驱动签名,Main-Class=服务端监控窗口
server.ui.ServerMonitorFrame);
给我一个 build-jars.sh:先 javac 全部源码,再分别 jar cfe;告诉我怎么 java -jar 启动。
```

② 让 AI 逐行解释打包脚本

```
用中文逐行解释这个 build-jars.sh:每一条命令在做什么。特别讲清楚:
javac 的 -cp 为什么要带那个 MySQL 驱动;jar cfe 的 c/f/e 各是什么意思;
第 ③ 步的 unzip 和那两行 rm 在干嘛(为什么合并驱动要删 META-INF 里的签名文件)。
```

③ 局域网演示 + 查 IP / 防火墙

```
教我局域网多台电脑演示:一台电脑起服务端(MySQL + 服务端 jar),其它电脑起客户端,
登录窗"服务器地址"填服务端电脑的局域网 IP。告诉我服务端电脑要注意什么。
```

```
我这台是 Windows(或 Mac),要当图书馆服务端。教我:① 用什么命令查我这台电脑的局域网 IP,
IPv4 那一行怎么认;② 怎么放行入站 TCP 9000,让局域网别的电脑能连进来。给我具体步骤。
```

④(可选)AI 图书助手

```
给客户端加一个"AI 图书助手"聊天面板:接大模型(用我自己的 key),
读者问"推荐一本 Java 的书 / 有没有《活着》",AI 通过 Function Calling 调服务端的
BOOK_QUERY 查库,
再用自然语言回答。定义一个 searchBooks 工具,两轮对话(第一轮带工具让模型决定调不调,
第二轮把查库结果喂回让它作答)。key 没配置时友好提示,不要崩溃。
```

十、Vibe Coding 全流程复盘(一张图)

```
反向提问理需求 → 写 Agent PRD(.trae/rules 图纸)
   → SOLO 分步生成(建库 → 服务端 → 登录链路 → 客户端 → 借还 → 管理端 → 监控窗口)
      → C/S 联调 → 纠错三步法 → 代码审查(信任但验证)→ 打包双 jar + 局域网上线
```

你真正学到的四件事(比敲代码更值钱):把需求说清楚(Agent PRD)· 指挥 AI 分步干活 · 判断 AI 写得对不对(信任但验证)· 让系统跑起来给人用(打包上线)。**你学的不是敲得快,是想得清、说得明、审得动。**

十一、FAQ

Q1:C/S 的"上线"到底和网页有什么不同? 网页上线是部署到服务器、浏览器访问;我们是桌面程序,"上线"= 打成 jar + 局域网多台电脑连同一个服务端。不用买服务器、不用浏览器。

Q2:jar 到底是什么?`java -jar` 背后发生了什么? jar 是把一堆编译出来的 `.class` 打成的一个文件(本质是 zip)。`java -jar` 会读 jar 里清单的 `Main-Class`,从那个类的 `main` 开始跑。所以入口(Main-Class)指错,就双击没反应 / 报 `no main manifest attribute`。

Q3:为什么服务端要 fat jar,客户端不用? 服务端要用 JDBC 连数据库,得带 MySQL 驱动(靠 classpath 找到),所以把驱动打进去(fat jar);客户端不直连库、只发 Socket,不需要任何第三方依赖,普通 jar 就行。

Q4:服务端 jar 一跑就报签名错怎么办? 合并驱动时没删签名。删掉解开的驱动里 `META-INF/*.SF`、`*.RSA`、`*.DSA`、`MANIFEST.MF` 和 `META-INF/versions` 再打包(保留 `META-INF/services/java.sql.Driver`)。这是打 fat jar 最经典的坑。

Q5:别的电脑连不上服务端,最可能是什么? 两件事优先查:① 客户端登录窗服务器地址是不是还填着 `127.0.0.1`(要填服务器电脑的 IP);② 服务器电脑防火墙是不是拦了 9000(放行入站 TCP 9000)。再确认两台在同一 WiFi。

Q6:怎么查我这台电脑的局域网 IP?为什么不能填 127.0.0.1? Windows `ipconfig` 看 IPv4、Mac `ifconfig | grep 192`,找 `192.168.x.x`。`127.0.0.1` 意思是"我自己这台",客户端填它就是"连我自己上的服务端",而客户端电脑上根本没跑服务端,所以连不上;局域网要填**服务器电脑的 IP**。

Q7:多台客户端同时借书,会不会把库存扣成负数? 不会。防超借靠**服务端**:一个事务里 `UPDATE ... SET stock=stock-1 WHERE id=? AND stock>0` 按影响行数判,只有扣到(影响行数=1)才算借成。业务集中在服务端把关,这正是档 B 的价值。

Q8:没有大模型 key,今天是不是就没法完成? 不是。AI 助手(Step 3)是**可选 / 挑战**,和今天的核心验收(打包双 jar + 局域网上线)无关。没 key 就跳过,照样拿到今天该拿的产出。

Q9:Part B 分组该怎么选档? 管理类(超市 POS / 酒店 / 停车场 / 二手交易,单机或少量客户端)可走**档 A 两层**(参考 `cs-swing-pos-demo`);通信 / 实时类(聊天室 / 对战 / 文件传输)走**档 B 推送型变体**(服务端还要主动广播,参考 `cs-swing-chatroom-demo`)。方法同一套,换的只是业务。

今日产出

从零补上了 **jar 打包**(jar / javac / Main-Class / classpath / fat jar / 去签名)和 **局域网部署**(局域网 / IP / 端口 / 防火墙 / 0.0.0.0)两块基础;把 C/S 图书馆**打成两个可执行 jar**(服务端 fat jar 含驱动去签名 + 客户端纯 Swing),做通**局域网多机上线**(填服务器 IP + 防火墙放行 9000),现场验证**并发防超借**,并把 Vibe Coding 全流程复盘成一张图。**代码写得再好,跑起来给人用,系统才真正活了——这就是你今天走完的最后一公里。**